Week 3 Lesson 2 — Decision Tree Algorithm, Explained

8th November 2026

1 Decision Tree Algorithm, Explained

Article found in Decision Tree Algorithm, Explained

1.1 Introduction

Classification is a two-step process, learning step and prediction step, in machine learning. In the learning step, the model is developed based on given training data. In the prediction step, the model is used to predict the response for given data. Decision Tree is one of the easiest and popular classification algorithms to understand and interpret.

1.2 Decision Tree Algorithm

Decision Tree algorithm belongs to the family of supervised learning algorithms. Unlike other supervised learning algorithms, the decision tree algorithm can be used for solving **regression and classification problems** too.

The goal of using a Decision Tree is to create a training model that can use to predict the class or value of the target variable by **learning simple decision rules** inferred from prior data (training data).

In Decision Trees, for predicting a class label for a record we start from the **root** of the tree. We compare the values of the root attribute with the record's attribute. On the basis of comparison, we follow the branch corresponding to that value and jump to the next node.

1.3 Types of Decision Trees

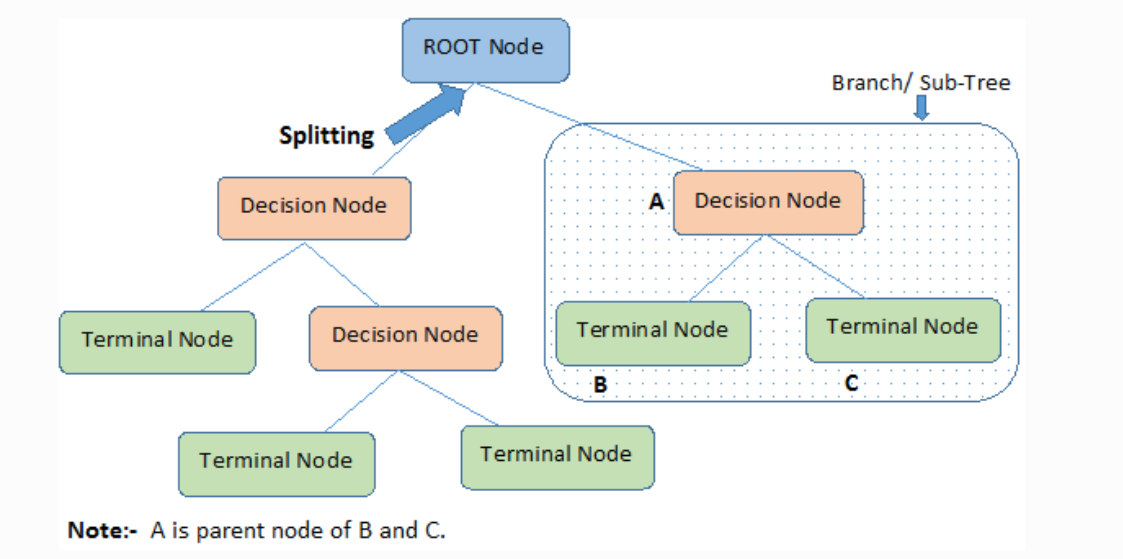
Types of decision trees are based on the type of target variable we have. It can be of two types:

1. **Categorical Variable Decision Tree:** Decision Tree which has a categorical target variable then it called a **Categorical variable decision tree.**
2. **Continuous Variable Decision Tree:** Decision Tree has a continuous target variable then it is called **Continuous Variable Decision Tree.**

Example: - Let's say we have a problem to predict whether a customer will pay his renewal premium with an insurance company (yes/ no). Here we know that the income of customers is a significant variable but the insurance company does not have income details for all customers. Now, as we know this is an important variable, then we can build a decision tree to predict customer income based on occupation, product, and various other variables. In this case, we are predicting values for the continuous variables.

1.4 Important Terminology related to Decision Trees

1. **Root Node:** It represents the entire population or sample and this further gets divided into two or more homogeneous sets.
2. **Splitting:** It is a process of dividing a node into two or more sub-nodes.
3. **Decision Node:** When a sub-node splits into further sub-nodes, then it is called the decision node.
4. **Leaf / Terminal Node:** Nodes do not split is called Leaf or Terminal node.
5. **Pruning:** When we remove sub-nodes of a decision node, this process is called pruning. You can say the opposite process of splitting.
6. **Branch / Sub-Tree:** A subsection of the entire tree is called branch or sub-tree.
7. **Parent and Child Node:** A node, which is divided into sub-nodes is called a parent node of sub-nodes whereas sub-nodes are the child of a parent node.



Decision trees classify the examples by sorting them down the tree from the root to some leaf/terminal node, with the leaf/terminal node providing the classification of the example.

Each node in the tree acts as a test case for some attribute, and each edge descending from the node corresponds to the possible answers to the test case. This process is recursive in nature and is repeated for every subtree rooted at the new node.

1.5 Assumptions while creating Decision Tree

Below are some of the assumptions we make while using Decision tree:

- In the beginning, the whole training set is considered as the **root.**
- Feature values are preferred to be categorical. If the values are continuous then they are discretized prior to building the model.
- Records are **distributed recursively** on the basis of attribute values.
- Order to placing attributes as root or internal node of the tree is done by using some statistical approach.

Decision Trees follow **Sum of Product (SOP)** representation. The Sum of product (SOP) is also

known as **Disjunctive Normal Form**. For a class, every branch from the root of the tree to a leaf node having the same class is conjunction (product) of values, different branches ending in that class form a disjunction (sum).

The primary challenge in the decision tree implementation is to identify which attributes do we need to consider as the root node and each level. Handling this is to know as the attributes selection. We have different attributes selection measures to identify the attribute which can be considered as the root note at each level.

1.6 How do Decision Trees work?

The decision of making strategic splits heavily affects a tree's accuracy. The decision criteria are different for classification and regression trees.

Decision trees use multiple algorithms to decide to split a node into two or more sub-nodes. The creation of sub-nodes increases the homogeneity of resultant sub-nodes. In other words, we can say that the purity of the node increases with respect to the target variable. The decision tree splits the nodes on all available variables and then selects the split which results in most homogeneous sub-nodes.

The algorithm selection is also based on the type of target variables. Let us look at some algorithms used in Decision Trees:

ID3 $\rightarrow$ (extension of D3) **C4.5** $\rightarrow$ (successor of ID3) **CART** $\rightarrow$ (Classification And Regression Tree) **CHAID** $\rightarrow$ (Chi-square automatic interaction detection Performs multi-level splits when computing classification trees) *MARS* $\rightarrow$ (multivariate adaptive regression splines)

The ID3 algorithm builds decision trees using a top-down greedy search approach through the space of possible branches with no backtracking. A greedy algorithm, as the name suggests, always makes the choice that seems to be the best at that moment.

1.7 Steps in ID3 algorithm:

1. It begins with the original set S as the root node.
2. On each iteration of the algorithm, it iterates through the very unused attribute of the set S and calculates **Entropy(H)** and **Information gain(IG)** of this attribute.
3. It then selects the attribute which has the smallest Entropy or Largest Information gain.
4. The set S is then split by the selected attribute to produce a subset of the data.
5. The algorithm continues to recur on each subset, considering only attributes never selected before.

1.8 Attribute Selection Measures

If the dataset consists of **N** attributes then deciding which attribute to place at the root or at different levels of the tree as internal nodes is a complicated step. By just randomly selecting any node to be the root can't solve the issue. If we follow a random approach, it may give us bad results with low accuracy.

For solving this attribute selection problem, researchers worked and devised some solutions. They suggested using some criteria like :

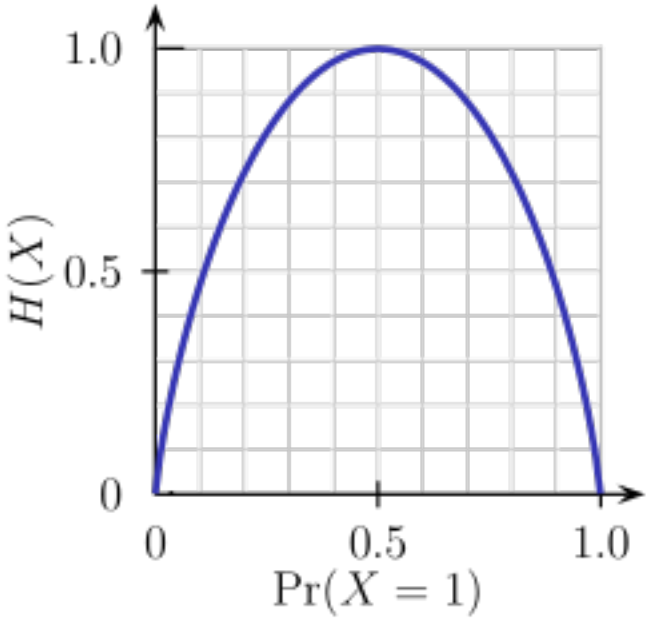
- Entropy,

- Information gain,
- Gini index,
- Gain Ratio,
- Reduction in Variance,
- Chi-Square

These criteria will calculate values for every attribute. The values are sorted, and attributes are placed in the tree by following the order i.e, the attribute with a high value(in case of information gain) is placed at the root. While using Information Gain as a criterion, we assume attributes to be categorical, and for the Gini index, attributes are assumed to be continuous.

1.9 Entropy

Entropy is a measure of the randomness in the information being processed. The higher the entropy, the harder it is to draw any conclusions from that information. Flipping a coin is an example of an action that provides information that is random.



From the above graph, it is quite evident that the entropy $H(X)$ is zero when the probability is either 0 or 1. The Entropy is maximum when the probability is 0.5 because it projects perfect randomness in the data and there is no chance if perfectly determining the outcome.

> ID3 follows the rule — A branch with an entropy of zero is a leaf node and A brach with entropy more than zero needs further splitting.

Mathematically Entropy for 1 attribute is represented as:

$$E(S) = \sum_{i=1}^{c} - p_i \log_2 p_i$$

Play Golf	
Yes	No
9	5



Entropy(PlayGolf) = Entropy (5,9)
= Entropy (0.36, 0.64)
= - (0.36 $\log_2$ 0.36) - (0.64 $\log_2$ 0.64)
= 0.94

Where **S → Current state, and Pi → Probability of an event i of state S or Percentage of class i in a node of state S.**

[1]:
```
import numpy as np

play_golf_dict = {'Yes': 9, 'No': 5}
total = sum(play_golf_dict.values())
print(f"Total: {total}")
play_golf_prob_dict = {
    outcome: count / total
    for outcome, count in play_golf_dict.items()
}
#print(play_golf_prob_dict)
for outcome, count in play_golf_prob_dict.items():
    print(f"Play Golf {outcome} = {count:.2f}")

entropy_play_golf = sum(-count * np.log2(count) for count in␣
 ↪play_golf_prob_dict.values())

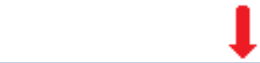
print(f"Entropy Play Golf = {entropy_play_golf:.2f}")
```

```
Total: 14
Play Golf Yes = 0.64
Play Golf No = 0.36
Entropy Play Golf = 0.94
```

Mathematically Entropy for multiple attributes is represented as:

$$E(T,X) = \sum_{c \in X} P(c)E(c)$$

		Play Golf		
		Yes	No	
Outlook	Sunny	3	2	5
	Overcast	4	0	4
	Rainy	2	3	5
				14



E(PlayGolf, Outlook) = **P**(Sunny)***E**(3,2) + **P**(Overcast)***E**(4,0) + **P**(Rainy)***E**(2,3)

= (5/14)*0.971 + (4/14)*0.0 + (5/14)*0.971

= 0.693

where **T→ Current state and X → Selected attribute**

[2]:
```
import pandas as pd

outlook_df = pd.DataFrame(
    {
        "Yes": [3, 4, 2],
        "No": [2, 0, 3],
    },
    index=pd.Index(["Sunny", "Overcast", "Rainy"], name="Outlook"),
)

outlook_df["Total"] = outlook_df.sum(axis=1)
outlook_df.loc["Total"] = outlook_df.sum()

outlook_df
```

[2]:
```
          Yes  No  Total
Outlook
Sunny       3   2      5
Overcast    4   0      4
Rainy       2   3      5
Total       9   5     14
```

[3]:
```
outlook_df.loc[["Sunny", "Overcast", "Rainy"], "Total"].values
```

[3]:
```
array([5, 4, 5])
```

[4]:
```
total_sum = outlook_df.loc["Total", "Total"]
entropy_play_golf = 0

print(f"Total Sum = {total_sum}")

for index, row in outlook_df.iterrows():
    if index == "Total":
        continue

    probabilities = row[["Yes", "No"]] / row["Total"]
    print(f"Probabilities for␣
 ↪{index}\n------------------------------\n{probabilities}")

    # Ignore zero probabilities: their entropy contribution is zero.
    probabilities = probabilities[probabilities > 0]
    row_entropy = -(probabilities * np.log2(probabilities)).sum()

    weight = row["Total"] / total_sum
    entropy_play_golf += weight * row_entropy

    print(f"{index}: weight = {weight:.3f} entropy = {row_entropy:.3f}")

print(f"\nWeighted entropy after splitting by Outlook = {entropy_play_golf:.
 ↪3f}")
```

```
Total Sum = 14
Probabilities for Sunny
------------------------------
Yes    0.6
No     0.4
Name: Sunny, dtype: float64
Sunny: weight = 0.357 entropy = 0.971
Probabilities for Overcast
------------------------------
Yes    1.0
No     0.0
Name: Overcast, dtype: float64
Overcast: weight = 0.286 entropy = -0.000
Probabilities for Rainy
------------------------------
Yes    0.4
No     0.6
Name: Rainy, dtype: float64
Rainy: weight = 0.357 entropy = 0.971
```

```
Weighted entropy after splitting by Outlook = 0.694
```

1.10 Information Gain

Information gain or **IG** is a statistical property that measures how well a given attribute separates the training examples according to their target classification. Constructing a decision tree is all about finding an attribute that returns the highest information gain and the smallest entropy.

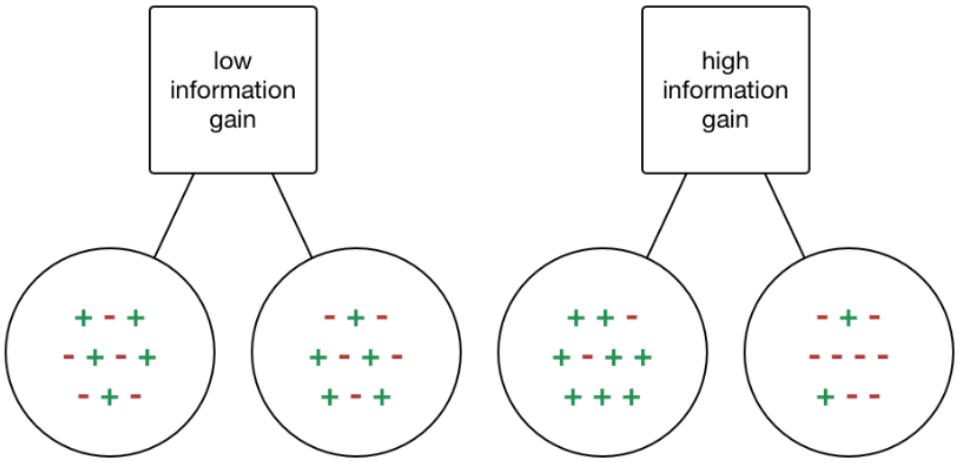


Image source: Information Gain

Information gain is a decrease in entropy. It computes the difference between entropy before split and average entropy after split of the dataset based on given attribute values. ID3 (Iterative Dichotomiser) decision tree algorithm uses information gain.

Mathematically, IG is represented as:

$$Information\,Gain(T, X) = Entropy(T) - Entropy(T, X)$$

$$IG(PlayGolf, Outlook) = E(PlayGolf) - E(PlayGolf, Outllook) = 0.940 - 0.693 = 0.247$$

In a much simpler way, we can conclude that:

$$Information\,Gain = Entropy(before) - \sum_{j=1}^{K} Entropy(j, after)$$

Information Gain

Where "before" is the dataset before the split, K is the number of subsets generated by the split, and $(j, after)$ is subset j after the split.

1.11 Gini Index

You can understand the Gini index as a cost function used to evaluate splits in the dataset. It is calculated by subtracting the sum of the squared probabilities of each class from one. It favors larger partitions and easy to implement whereas information gain favors smaller partitions with distinct values.

$$\text{Gini} = 1 - \sum_{i=1}^{C} (p_i)^2$$

Gini Index works with the categorical target variable "Success" or "Failure". It performs only Binary splits.

> Higher value of Gini index implies higher inequality, higher heterogeneity.

1.11.1 Steps to Calculate Gini index for a split

1. Calculate Gini for sub-nodes, using the above formula for success (p) and failure (q) (p^2+q^2).
2. Calculate the Gini index for split using the weighted Gini score of each node of that split.
3. CART (Classification and Regression Tree) uses the Gini index method to create split points.

1.12 Gain ratio

Information gain is biased towards choosing attributes with a large number of values as root nodes. It means it prefers the attribute with a large number of distinct values.

C4.5, an improvement of ID3, uses Gain ratio which is a modification of Information gain that reduces its bias and is usually the best option. Gain ratio overcomes the problem with information gain by taking into account the number of branches that would result before making the split. It corrects information gain by taking the intrinsic information of a split into account.

> Let us consider if we have a dataset that has users and their movie genre preferences based on variables like gender, group of age, rating, blah, blah. With the help of information gain, you split at 'Gender' (assuming it has the highest information gain) and now the variables 'Group of Age' and 'Rating' could be equally important and with the help of gain ratio, it will penalize a variable with more distinct values which will help us decide the split at the next level.

$$\text{GainRatio} = \frac{\text{InformationGain}}{\text{SplitInfo}} = \frac{\text{Entropy(before)} - \sum_{j=1}^{K} \text{Entropy}(j, \text{after})}{\sum_{j=1}^{K} w_j \log_2 w_j}$$

Gain Ratio

Where "before" is the dataset before the split, K is the number of subsets generated by the split, and (j, after) is subset j after the split.

1.13 Reduction in Variance

Reduction in variance is an algorithm used for continuous target variables (regression problems). This algorithm uses the standard formula of variance to choose the best split. The split with lower variance is selected as the criteria to split the population:

$$\text{Variance} = \frac{\sum(X_i - \bar{X})^2}{n}$$

Above X-bar is the mean of the values, X is actual and n is the number of values.

1.13.1 Steps to calculate Variance:

1. Calculate variance for each node.
2. Calculate variance for each split as the weighted average of each node variance.

1.14 Chi-Square

The acronym CHAID stands for Chi-squared Automatic Interaction Detector. It is one of the oldest tree classification methods. It finds out the statistical significance between the differences between sub-nodes and parent node. We measure it by the sum of squares of standardized differences between observed and expected frequencies of the target variable.

It works with the categorical target variable "Success" or "Failure". It can perform two or more splits. Higher the value of Chi-Square higher the statistical significance of differences between sub-node and Parent node.

It generates a tree called CHAID (Chi-square Automatic Interaction Detector).

Mathematically, Chi-squared is represented as:

$$\chi^2 = \sum \frac{(O - E)^2}{E}$$

Where:

- χ^2 = chi-square statistic
- $\sum$ = the sum over all categories
- O = observed frequency
- E = expected frequency

1.14.1 Steps to Calculate Chi-square for a split:

1. Calculate Chi-square for an individual node by calculating the deviation for Success and Failure both
2. Calculated Chi-square of Split using Sum of all Chi-square of success and Failure of each node of the split

1.15 How to avoid/counter Overfitting in Decision Trees?

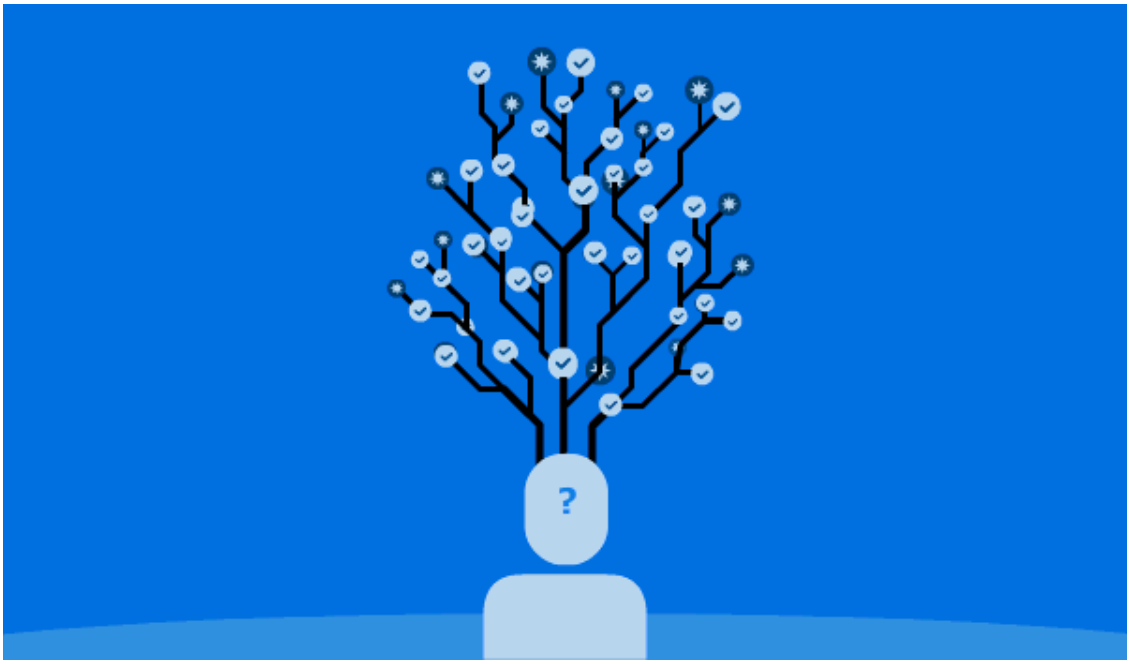
The common problem with Decision trees, especially having a table full of columns, they fit a lot. Sometimes it looks like the tree memorized the training data set. If there is no limit set on a decision tree, it will give you 100% accuracy on the training data set because in the worse case it will end up making 1 leaf for each observation. Thus this affects the accuracy when predicting samples that are not part of the training set.

Here are two ways to remove overfitting:

1. Pruning Decision Trees.
2. Random Forest

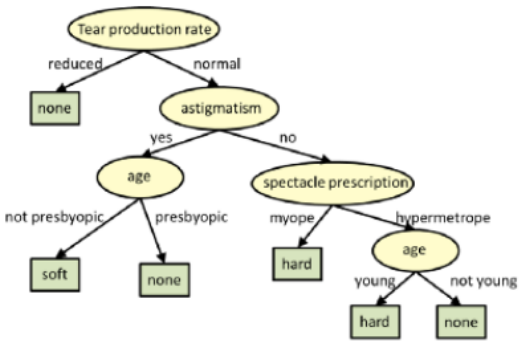
1.16 Pruning Decision Trees

The splitting process results in fully grown trees until the stopping criteria are reached. But, the fully grown tree is likely to overfit the data, leading to poor accuracy on unseen data.

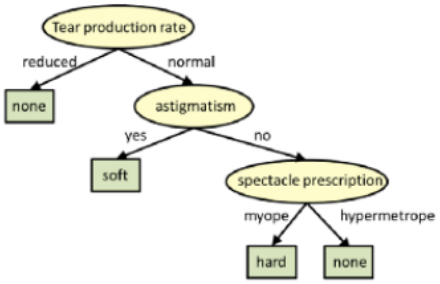


Pruning in action

In **pruning**, you trim off the branches of the tree, i.e., remove the decision nodes starting from the leaf node such that the overall accuracy is not disturbed. This is done by segregating the actual training set into two sets: training data set, D and validation data set, V. Prepare the decision tree using the segregated training data set, D. Then continue trimming the tree accordingly to optimize the accuracy of the validation data set, V.



Original Tree



Pruned Tree

Pruning

In the above diagram, the 'Age' attribute in the left-hand side of the tree has been pruned as it has more importance on the right-hand side of the tree, hence removing overfitting.

1.17 Random Forest

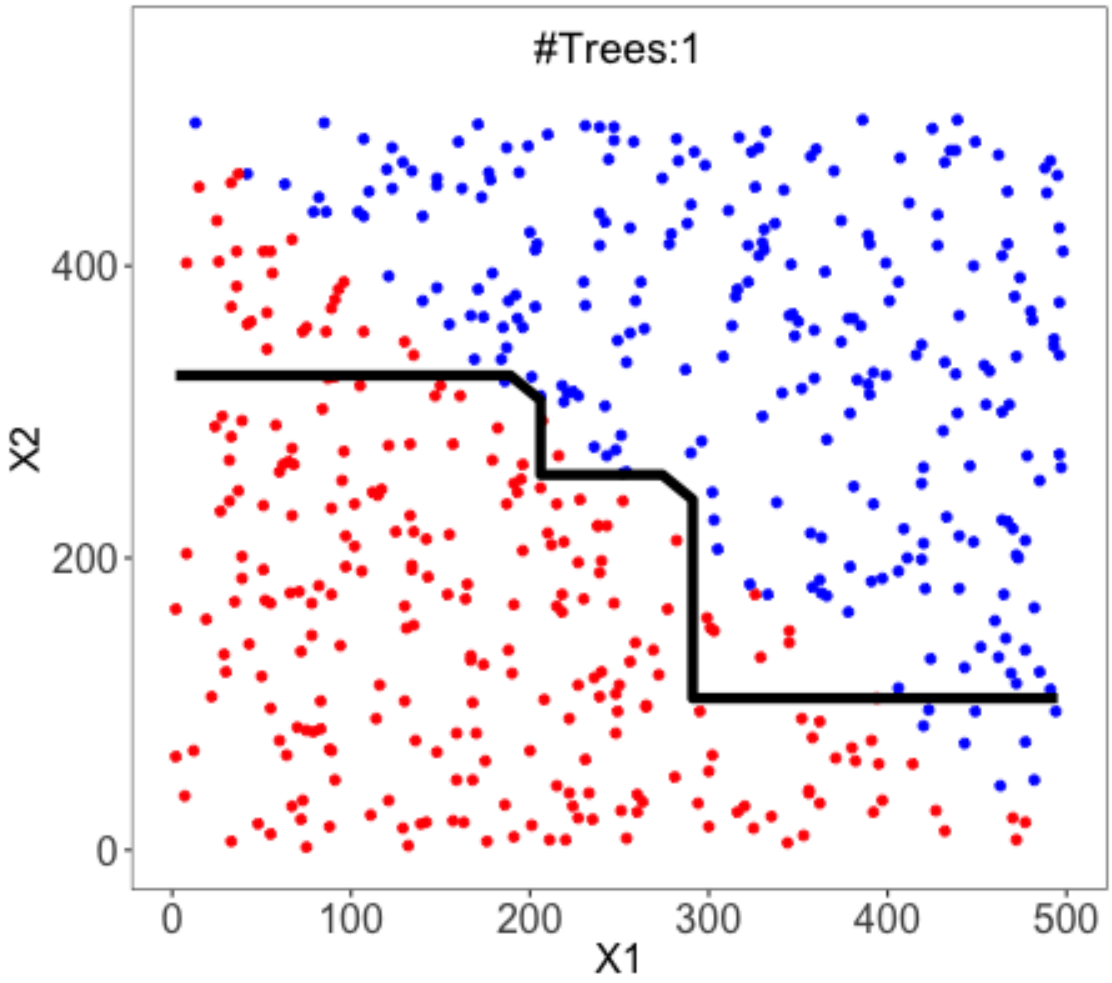
Random Forest is an example of ensemble learning, in which we combine multiple machine learning algorithms to obtain better predictive performance.

Why the name "Random"?

Two key concepts that give it the name random:

A random sampling of training data set when building trees. Random subsets of features considered when splitting nodes. A technique known as bagging is used to create an ensemble of trees where multiple training sets are generated with replacement.

In the bagging technique, a data set is divided into N samples using randomized sampling. Then, using a single learning algorithm a model is built on all samples. Later, the resultant predictions are combined using voting or averaging in parallel.



Random Forest in action

1.18 Which is better Linear or tree-based models?

Well, it depends on the kind of problem you are solving.

1. If the relationship between dependent & independent variables is well approximated by a linear model, linear regression will outperform the tree-based model.
2. If there is a high non-linearity & complex relationship between dependent & independent variables, a tree model will outperform a classical regression method.
3. If you need to build a model that is easy to explain to people, a decision tree model will always do better than a linear model. Decision tree models are even simpler to interpret than linear regression!

1.19 Additional images

1.19.1 Entropy

This plot shows **binary entropy**: how uncertain the class label is when there are two possible classes.

Let **p** be the probability of class 1. The probability of class 0 is **1 - p**, because the two probabilities must add to 1.

$$H(p) = -p\log_2(p) - (1-p)\log_2(1-p)$$

Notice the brackets: the second term is **-(1 - p) × log2(1 - p)**, not **-(1 - p × log2(p))**. Each class uses its own probability both outside and inside the logarithm.

Why this formula?

- **-log2(probability)** measures the information, or surprise, of an outcome. A certain outcome gives 0 bits; an outcome with probability 1/2 gives 1 bit; one with probability 1/4 gives 2 bits. Rarer outcomes are more surprising. Logarithms make information add when independent outcomes occur together.
- Multiplying each surprise by its probability gives its contribution to the **average surprise**. Entropy adds these contributions for both classes: p × [-log2(p)] + (1 - p) × [-log2(1 - p)].
- For probabilities between 0 and 1, the logarithm is negative. The minus signs make the contributions positive. Base 2 expresses the result in **bits**.

Examples

p	1 - p	Entropy	Meaning
0	1	0 bits	Always class 0: no uncertainty
0.5	0.5	1 bit	Both classes equally likely: maximum binary uncertainty
1	0	0 bits	Always class 1: no uncertainty

At p = 0.5, H = -0.5 × (-1) - 0.5 × (-1) = 1 bit. At p = 0.9 (or 0.1), H is about 0.469 bits: one class is more predictable, so uncertainty is lower.

How the NumPy code works

- p contains the probabilities to plot, from 0 to 1. entropy starts as an array of zeros.
- mask = (p > 0) & (p < 1) selects only probabilities strictly between 0 and 1. The & operator combines the two comparisons element by element.
- p[mask] selects those probabilities, and entropy[mask] stores their calculated entropy.
- The mask avoids evaluating log2(0) at the endpoints. Mathematically, the contribution -q × log2(q) approaches 0 as q approaches 0, so we define that contribution as 0. The endpoint entropy values therefore correctly remain at their initialized value of 0.

In a decision tree, p is the proportion of one class at a node. Low entropy means the node is mostly one class; high entropy means it is mixed. An entropy-based tree chooses splits that reduce the weighted average entropy of the child nodes (maximize information gain).

[5]:
```
import numpy as np
import matplotlib.pyplot as plt
```

```
p = np.linspace(0, 1, 501)
entropy = np.zeros_like(p)

# At p = 0 and p = 1, entropy is zero; avoid log2(0).
mask = (p > 0) & (p < 1)
entropy[mask] = (
    -p[mask] * np.log2(p[mask])
    -(1 - p[mask]) * np.log2(1 - p[mask])
)

fig, ax = plt.subplots(figsize=(3, 3), facecolor="white")
ax.set_facecolor("white")

ax.plot(p, entropy, color="#4035c8", linewidth=2)
ax.set(xlim=(0, 1), ylim=(0, 1))
ax.set_box_aspect(1)

# Square gray grid, without visible tick labels.
ticks = np.linspace(0, 1, 11)
ax.set_xticks(ticks)
ax.set_yticks(ticks)
ax.grid(color="#bdbdbd", linewidth=1)
ax.set_axisbelow(True)

ax.set_xlim(0, 1)
ax.set_ylim(0, 1.05)

ticks = [0, 0.5, 1.0]
labels = ["0", "0.5", "1.0"]

ax.set_xticks(ticks, labels=labels)
ax.set_yticks(ticks, labels=labels)

ax.set_xlabel("Pr(X=1)", color="black")
ax.set_ylabel("H(X)", color="black")

for spine in ax.spines.values():
    spine.set_visible(False)

labels = ["0", "0.5", "1.0"]
ax.set_xticks([0, 0.5, 1.0], labels=labels)
ax.set_yticks([0, 0.5, 1.0], labels=labels)

ax.set_xticks(np.linspace(0, 1, 11), minor=True)
ax.set_yticks(np.linspace(0, 1, 11), minor=True)
```

```
ax.grid(which="both", color="#bdbdbd", linewidth=0.7)
#ax.tick_params(which="both", length=0)
ax.tick_params(
    axis="both", which="major",
    length=6, width=1.2, colors="black", direction="out",
    bottom=True, left=True, top=False, right=False
)
ax.tick_params(axis="both", which="minor", length=0)
ax.set_axisbelow(True)

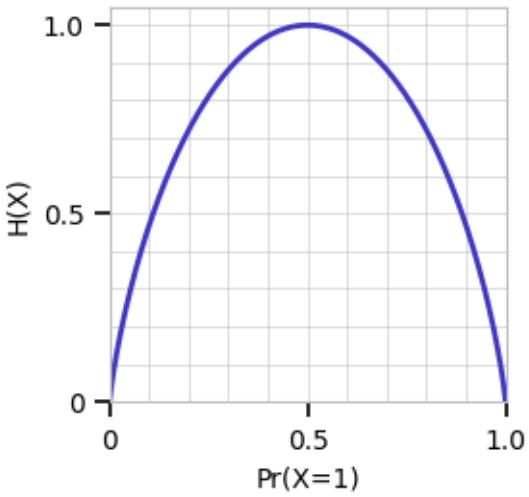
for spine in ax.spines.values():
    spine.set_color("#bdbdbd")
    spine.set_visible(True)

labels = ["0", "0.5", "1.0"]
ax.set_xticks([0, 0.5, 1.0], labels=labels)
ax.set_yticks([0, 0.5, 1.0], labels=labels)

ax.set_xticks(np.linspace(0, 1, 11), minor=True)
ax.set_yticks(np.linspace(0, 1, 11), minor=True)

ax.grid(which="both", color="#bdbdbd", linewidth=0.7)
ax.set_axisbelow(True)

plt.tight_layout()
plt.show()
```



1.19.2 Logarithm properties

Logarithmic Properties	
Product Rule	$\log_a(xy) = \log_a x + \log_a y$
Quotient Rule	$\log_a\left(\frac{x}{y}\right) = \log_a x - \log_a y$
Power Rule	$\log_a x^p = p \log_a x$
Change of Base Rule	$\log_a x = \frac{\log_b x}{\log_b a}$
Equality Rule	If $\log_a x = \log_a y$ then $x = y$

Source: Online Math Learning

1.20 Load all the basic libraries.

1.20.1 About the dataset

"Supermarket Sales Dataset: This dataset captures detailed transaction records from a supermarket chain in Myanmar across three cities: Yangon, Naypyitaw, and Mandalay. It provides a comprehensive view of sales activities, customer demographics, and payment methods from January to March 2019.

File Information:

- File Format: CSV (Comma Separated Values)
- File Size: Approximately X KB (replace X with actual size)
- Number of Rows (Records): 1000 (replace with your dataset size)
- Number of Columns: 17

Column Descriptions:

- Invoice ID: A unique identifier for each transaction.
- Branch: The branch location of the supermarket (e.g., Yangon, Naypyitaw, Mandalay).
- City: The city where the branch is located.
- Customer Type: 'Member' or 'Normal' customer.
- Gender: Gender of the customer.
- Product Line: Product category (e.g., Health & Beauty, Electronic Accessories).
- Unit Price: Price per unit of the product.
- Quantity: Number of items purchased.

- Tax 5%: Tax amount based on a 5% rate.
- Total: Total transaction amount including tax.
- Date: Date of the transaction.
- Time: Time of the transaction.
- Payment: Payment method (Cash, Ewallet, Credit card).
- COGS (Cost of Goods Sold): Raw cost of the products sold.
- Gross Margin Percentage: Set at 4.7619%.
- Gross Income: Profit from each transaction.
- Customer Rating: Customer satisfaction rating (out of 10).

This dataset is well-suited for sales analysis, customer behavior studies, and predictive modeling. It can be used for tasks such as customer segmentation, trend analysis, and forecasting."

[6]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from pathlib import Path
from urllib.request import urlretrieve

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from sklearn.tree import DecisionTreeClassifier
from sklearn import metrics

from matplotlib.colors import ListedColormap

# Dataset
url = "https://huggingface.co/datasets/Rodrigopiva/Social_Network_Ads.csv/
 ↪resolve/main/Social_Network_Ads.csv"
file_path = Path("data/week_3_Social_Network_Ads.csv")

file_path.parent.mkdir(parents=True, exist_ok=True)

if not file_path.exists():
    urlretrieve(url, file_path)
    print(f"Downloaded the dataset: {file_path}")
else:
    print(f"Dataset already exists: {file_path}")
```

```
Dataset already exists: data/week_3_Social_Network_Ads.csv
```

Load the dataset. It consists of 5 features, UserID, Gender, Age, EstimatedSalary and Purchased.

Note: in the original article, the file is called 'Social.csv', but the name is irrelevant.

[7]:
```
data = pd.read_csv(file_path)
data.head()
```

[7]:

	User ID	Gender	Age	EstimatedSalary	Purchased
0	15624510	Male	19	19000	0
1	15810944	Male	35	20000	0
2	15668575	Female	26	43000	0
3	15603246	Female	27	57000	0
4	15804002	Male	19	76000	0

We will take only `Age` and `EstimatedSalary` as our independent variables X because of other features like `Gender` and 'User ID are irrelevant and have no effect on the purchasing capacity of a person. Purchased is our dependent variable y.

[8]:
```
feature_cols = ['Age','EstimatedSalary' ]
X = data.iloc[:,[2,3]].values
y = data.iloc[:,4].values
```

The next step is to split the dataset into training and test.

[9]:
```
random_state=0
X_train, X_test, y_train, y_test =  train_test_split(X,y,test_size = 0.25,␣
 ↪random_state= random_state)
```

Perform feature scaling

Note: Feature scaling with `StandardScaler` is not required for `DecisionTreeClassifier`. Decision trees split features using thresholds rather than distances. Standardization preserves the ordering of feature values, so it generally produces the same splits. The scaling step below is optional for this model. Probably a typo from the blog's author.

[10]:
```
# The author uses feature scaling, but in reality it's not required.

#feature scaling
sc_X = StandardScaler()
X_train_scaled = sc_X.fit_transform(X_train)
X_test_scaled = sc_X.transform(X_test)
```

Fit the model in the Decision Tree classifier.

[11]:
```
classifier_scaled = DecisionTreeClassifier(random_state=random_state)
classifier_scaled.fit(X_train_scaled, y_train)

classifier_not_scaled = DecisionTreeClassifier(random_state=random_state)
classifier_not_scaled.fit(X_train, y_train)
```

[11]:
```
DecisionTreeClassifier(random_state=0)
```

Make predictions and check accuracy.

[12]:
```
#prediction
y_pred_scaled = classifier_scaled.predict(X_test_scaled)
y_pred_not_scaled = classifier_not_scaled.predict(X_test)
```

```
print(
    f"Accuracy Score when scaled: {metrics.accuracy_score(y_test,␣
 ↪y_pred_scaled):.3f}"
)
print(
    f"Accuracy Score when not scaled: {metrics.accuracy_score(y_test,␣
 ↪y_pred_not_scaled):.3f}"
)
print(
    "Number of differing predictions:",
    np.count_nonzero(y_pred_scaled != y_pred_not_scaled),
)
```

```
Accuracy Score when scaled: 0.900
Accuracy Score when not scaled: 0.900
Number of differing predictions: 0
```

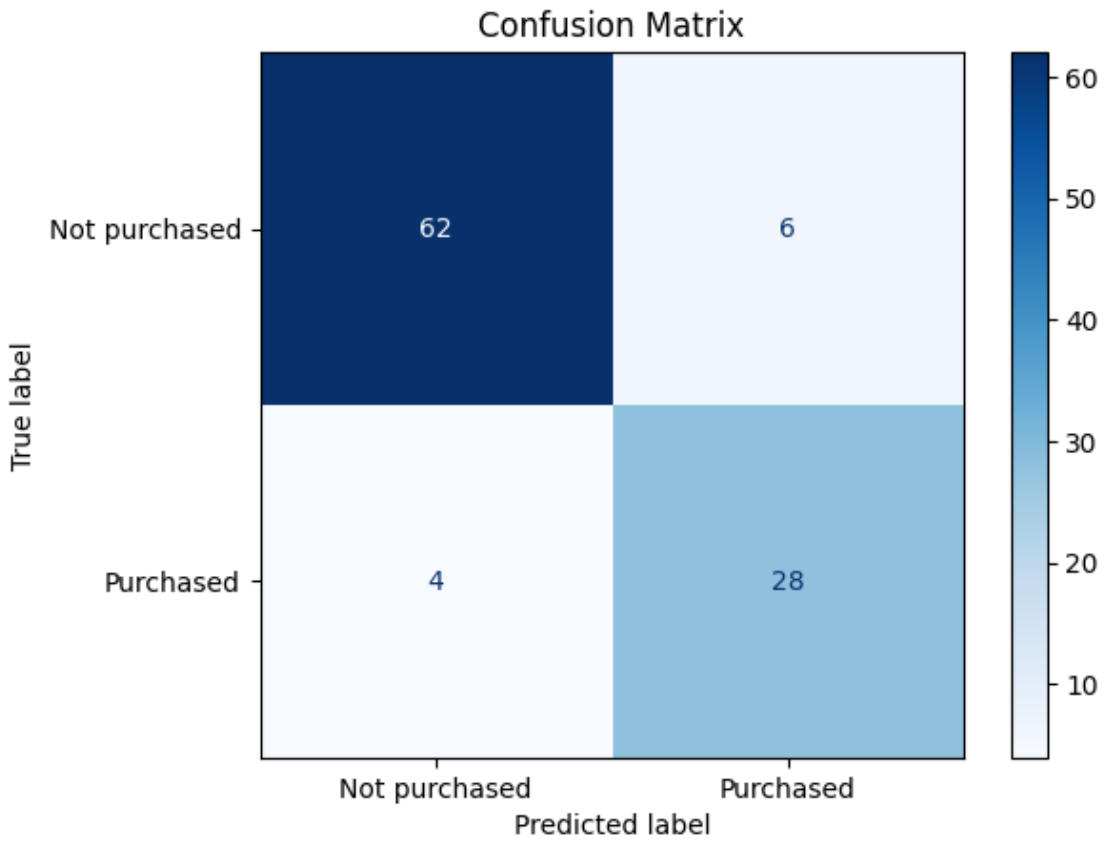
The decision tree classifier gave an accuracy of 91%.

Confusion Matrix

[13]:
```
cm = confusion_matrix(y_test, y_pred_scaled)

ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Not purchased", "Purchased"],
).plot(cmap="Blues", values_format="d")

plt.title("Confusion Matrix")
plt.show()
```



It means 6 observations have been classified as false.

Let us first visualize the model prediction results.

[14]:
```
'''X_set, y_set = X_test, y_test
X1, X2 = np.meshgrid(np.arange(start = X_set[:,0].min()-1, stop= X_set[:,0].
 ↪max()+1, step = 0.01),np.arange(start = X_set[:,1].min()-1, stop= X_set[:,1].
 ↪max()+1, step = 0.01))
plt.contourf(X1,X2, classifier_not_scaled.predict(np.array([X1.ravel(), X2.
 ↪ravel()]).T).reshape(X1.shape), alpha=0.75, cmap =␣
 ↪ListedColormap(("red","green")))
plt.xlim(X1.min(), X1.max())
plt.ylim(X2.min(), X2.max())
for i,j in enumerate(np.unique(y_set)):
    #plt.scatter(X_set[y_set==j,0],X_set[y_set==j,1], c =␣
 ↪ListedColormap(("red","green"))(i),label = j)
    plt.scatter(
        X_set[y_set == j, 0],
        X_set[y_set == j, 1],
        color=ListedColormap(("red", "green"))(i),
```

```
        label=j,
    )
plt.title("Decision Tree(Test set)")
plt.xlabel("Age")
plt.ylabel("Estimated Salary")
plt.legend()
plt.show()'''

def visualise_prediction_results(classifier, scaler=None, scale_x_axis=False,␣
 ↪scale_y_axis=False):
    """Plot raw X_test values; optionally scale either axis using scaler.

    Both axis options are ignored when scaler is None. Predictions always use
    the feature units expected by the classifier.
    """
    X_set, y_set = X_test, y_test
    scaled = scaler is not None

    # Build the prediction grid in original units.
    padding = np.ptp(X_set, axis=0) * 0.05
    padding = np.maximum(padding, 1)
    lower = X_set.min(axis=0) - padding
    upper = X_set.max(axis=0) + padding

    # Match the original min - 1 / max + 1 margins in scaled units.
    show_scaled_x = scaled and scale_x_axis
    show_scaled_y = scaled and scale_y_axis
    if show_scaled_x or show_scaled_y:
        scaled_points = scaler.transform(X_set)
        scaled_bounds = np.array([
            scaled_points.min(axis=0) - 1,
            scaled_points.max(axis=0) + 1,
        ])
        raw_bounds = scaler.inverse_transform(scaled_bounds)
        for axis, show_scaled in enumerate((show_scaled_x, show_scaled_y)):
            if show_scaled:
                lower[axis], upper[axis] = raw_bounds[:, axis]

    X1, X2 = np.meshgrid(
        np.linspace(
            lower[0],
            upper[0],
            300,
        ),
        np.linspace(
            lower[1],
            upper[1],
```

```
            300,
        ),
    )

    grid = np.column_stack((X1.ravel(), X2.ravel()))

    # Match the inputs to how this classifier was trained.
    prediction_inputs = scaler.transform(grid) if scaled else grid
    predictions = classifier.predict(prediction_inputs).reshape(X1.shape)

    # Change display coordinates only after computing model predictions.
    X_plot = X_set.astype(float, copy=True)
    if show_scaled_x or show_scaled_y:
        if show_scaled_x:
            X1 = prediction_inputs[:, 0].reshape(X1.shape)
            X_plot[:, 0] = scaled_points[:, 0]
        if show_scaled_y:
            X2 = prediction_inputs[:, 1].reshape(X2.shape)
            X_plot[:, 1] = scaled_points[:, 1]

    fig, ax = plt.subplots()
    cmap = ListedColormap(("red", "green"))

    ax.contourf(X1, X2, predictions, alpha=0.75, cmap=cmap)

    for i, label in enumerate(np.unique(y_set)):
        ax.scatter(
            X_plot[y_set == label, 0],
            X_plot[y_set == label, 1],
            color=cmap(i),
            label=label,
        )

    status = "Scaled features" if scaled else "Unscaled features"
    ax.set(
        title=f"Decision Tree (Test set) - {status}",
        xlabel="Age (scaled)" if show_scaled_x else "Age",
        ylabel="Estimated Salary (scaled)" if show_scaled_y else "Estimated␣
↪Salary",
        xlim=(X1.min(), X1.max()),
        ylim=(X2.min(), X2.max()),
    )
    ax.legend()
    plt.show()

# Set either option to False for original units; ignored without a scaler.
scale_x_axis = True
```

```
scale_y_axis = True
visualise_prediction_results(
    classifier_scaled, scaler=sc_X,
    scale_x_axis=True, scale_y_axis=True,
)
visualise_prediction_results(
    classifier_scaled, scaler=sc_X,
    scale_x_axis=False, scale_y_axis=False,
)
visualise_prediction_results(classifier_not_scaled)
```




Decision Tree (Test set) — Scaled features
0
1
140000
120000
100000
80000
60000
40000
20000
Estimated Salary
20
30
40
50
60
Age



Let us also visualize the tree:

You can use Scikit-learn's export_graphviz function to display the tree within a Jupyter notebook. For plotting trees, you also need to install Graphviz and pydotplus.

```
conda install python-graphviz
pip install pydotplus
```

'export_graphviz function converts decision tree classifier into dot file and pydotplus convert this dot file to png or displayable form on Jupyter.

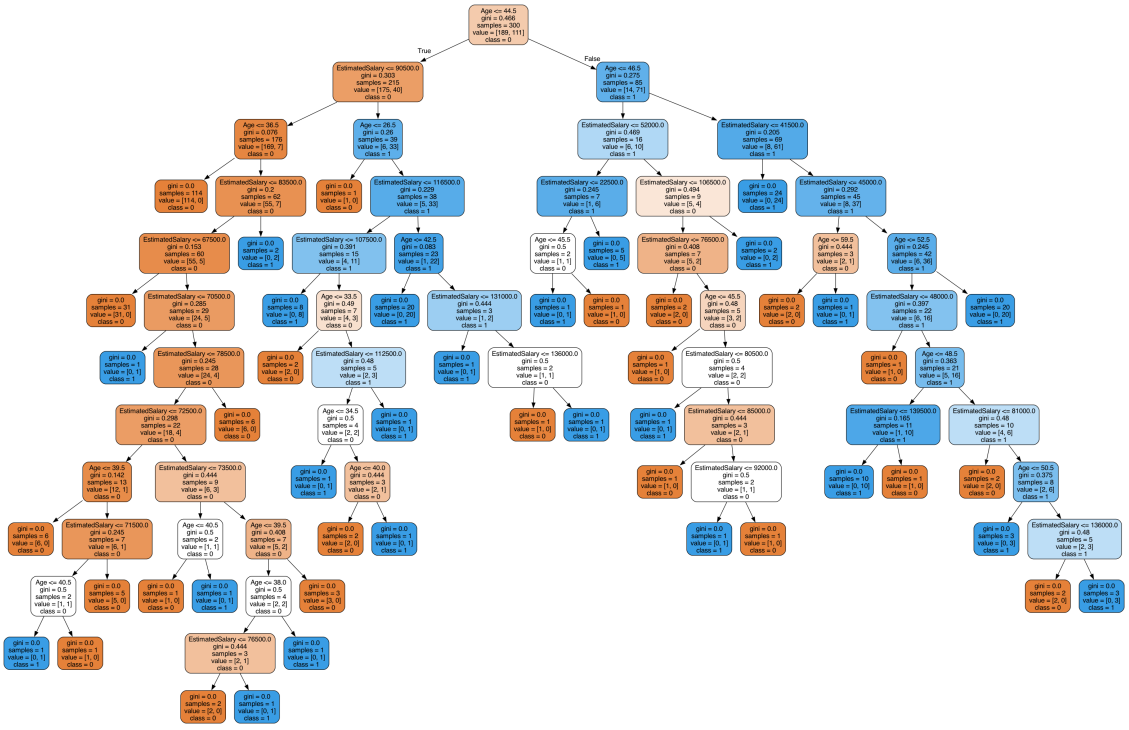
[15]:
```
use_sklearn_to_plot_tree_bool = False
classifier = classifier_not_scaled

if use_sklearn_to_plot_tree_bool:
    from sklearn.tree import plot_tree

    plt.figure(figsize=(20, 10))
    plot_tree(
        classifier,
        feature_names=feature_cols,
        class_names=["0", "1"],
```

```
        filled=True,
        rounded=True,
        fontsize=8,
    )
    plt.show()
else:
    from sklearn.tree import export_graphviz
    from io import StringIO # Deprecated: from sklearn.externals.six import␣
 ↪StringIO
    from IPython.display import Image
    import pydotplus

    # special_characters=True doesn't work. Set to false
    dot_data = StringIO()
    export_graphviz(classifier, out_file=dot_data,
                    filled=True, rounded=True,
                    special_characters=False,feature_names =␣
 ↪feature_cols,class_names=['0','1'])
    #graph = pydotplus.graph_from_dot_data(dot_data.getvalue())
    #Image(graph.create_png())
    dot_text = dot_data.getvalue().replace("\n", "") # This removes actual␣
 ↪newlines from the DOT source while preserving the escaped \n used inside␣
 ↪node labels. Keep special_characters=False.
    graph = pydotplus.graph_from_dot_data(dot_text)
    display(Image(graph.create_png()))
```



In the decision tree chart, each internal node has a decision rule that splits the data. Gini referred to as the Gini ratio, which measures the impurity of the node. You can say a node is pure when all of its records belong to the same class, such nodes known as the leaf node.

Here, the resultant tree is unpruned. This unpruned tree is unexplainable and not easy to understand. In the next section, let's optimize it by pruning.

1.21 Optimizing the Decision Tree Classifier

criterion: optional (default="gini") or Choose attribute selection measure: This parameter allows us to use the different-different attribute selection measure. Supported criteria are "gini" for the Gini index and "entropy" for the information gain.

splitter: string, optional (default="best") or Split Strategy: This parameter allows us to choose the split strategy. Supported strategies are "best" to choose the best split and "random" to choose the best random split.

max_depth: int or None, optional (default=None) or Maximum Depth of a Tree: The maximum depth of the tree. If None, then nodes are expanded until all the leaves contain less than min_samples_split samples. The higher value of maximum depth causes overfitting, and a lower value causes underfitting (Source).

In Scikit-learn, optimization of decision tree classifier performed by only pre-pruning. The maximum depth of the tree can be used as a control variable for pre-pruning.

[16]:
```
# Create Decision Tree classifer object
# the author forgot to put a random_state, I will use the one previously used.
# The results are the same wether scaled or not scaled.
classifier_scaled = DecisionTreeClassifier(criterion="entropy", max_depth=3,␣
 ↪random_state=random_state)# Train Decision Tree Classifer
classifier_scaled.fit(X_train_scaled,y_train)#Predict the response for test␣
 ↪dataset
y_pred_scaled = classifier.predict(X_test_scaled)# Model Accuracy, how often is␣
 ↪the classifier correct?
print("Accuracy when scaled:",metrics.accuracy_score(y_test, y_pred_scaled))

classifier_not_scaled = DecisionTreeClassifier(criterion="entropy",␣
 ↪max_depth=3, random_state=random_state)# Train Decision Tree Classifer
classifier_not_scaled.fit(X_train,y_train)#Predict the response for test dataset
y_pred_not_scaled = classifier_not_scaled.predict(X_test)# Model Accuracy, how␣
 ↪often is the classifier correct?
print("Accuracy when not scaled:",metrics.accuracy_score(y_test,␣
 ↪y_pred_not_scaled))
```

```
Accuracy when scaled: 0.68
Accuracy when not scaled: 0.94
```

Well, the classification rate increased to 94%, which is better accuracy than the previous model.

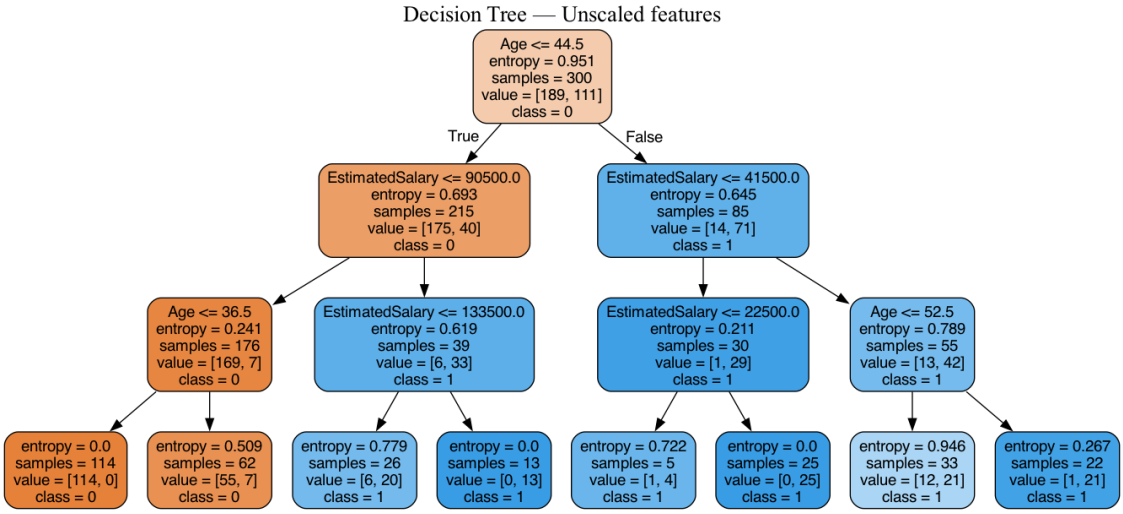
Now let us again visualize the pruned Decision tree after optimization.

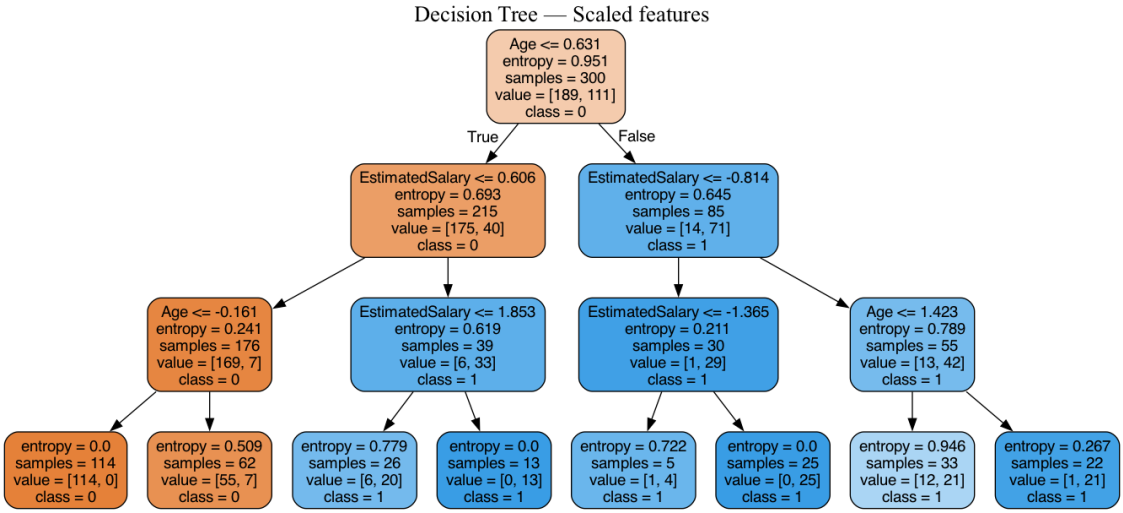
[17]:
```
if use_sklearn_to_plot_tree_bool:
    classifier = classifier_scaled
    plt.figure(figsize=(20, 10))
    plot_tree(
        classifier,
        feature_names=feature_cols,
        class_names=["0", "1"],
        filled=True,
        rounded=True,
        fontsize=8,
    )
    plt.show()
else:
    def plot_tree_classifier(classifier, title):
        dot_data = StringIO()
        export_graphviz(classifier, out_file=dot_data,
                        filled=True, rounded=True,
                        special_characters=False, feature_names =␣
 ↪feature_cols,class_names=['0','1'])
        #graph = pydotplus.graph_from_dot_data(dot_data.getvalue())
        dot_text = dot_data.getvalue().replace("\n", "") # This removes actual␣
 ↪newlines from the DOT source while preserving the escaped \n used inside␣
 ↪node labels. Keep special_characters=False.
        graph = pydotplus.graph_from_dot_data(dot_text)

        graph.set_label(title)
        graph.set_labelloc("t")  # Title at the top
        graph.set_fontsize(20)

        display(Image(graph.create_png()))

    plot_tree_classifier(classifier_not_scaled, "Decision Tree - Unscaled␣
 ↪features")
    plot_tree_classifier(classifier_scaled, "Decision Tree - Scaled features")
```





This pruned model is less complex, explainable, and easy to understand than the previous decision tree model plot.